

λ_p : Paths through Dependent Pairs

Anonymous Author(s)

Abstract

We present λ_p , a calculus in which paths traverse immutable dependent pairs. A pair member may depend on the first component and may contain either a term reference or a type definition. The calculus distinguishes term singleton types from abstract type selections. Conflating the two makes singleton symmetry unsound: we give a closed, well-typed program that becomes stuck. We adopt a conservative rule for dependent-pair subtyping and prove finite-run type safety in Lean. The unrestricted pair rule remains an open question for the syntax-separated calculus.

Keywords: path-dependent types, dependent pairs, singleton types, type soundness, Lean

1 Introduction

A path-dependent type assigns a type component to a stable term path. The usual object-calculus presentation combines this mechanism with recursive objects, refinements, intersections, and a substantial subtyping relation [1, 5]. The purpose of λ_p is narrower: to isolate paths, abstract type members, and dependency between the two components of a pair.

Values in λ_p are functions and immutable dependent pairs. A pair contains a first component and either a term reference or a type definition; the member signature may mention the first component. Paths follow first projections and named members through chains of pairs. A missed label resumes the search in the first component, making record lookup a direct traversal of the pair chain.

For example, let $r_1 = \langle b; a = z \rangle$ and $r_2 = \langle r_1; A = T \rangle$. Selection $r_2.A$ hits the outer type member, whereas $r_2.a$ misses A and continues as $r_2.1.a$, eventually returning z . The signature of the outer member may refer to its binder for r_1 , so the same pair structure supports both record extension and dependency on the preceding component.

Two design choices carry the metatheoretic weight. First, the singleton type $\{p\}$, which expresses value identity, is distinct from the abstract selection $p.A$, which denotes a type between declared bounds. An earlier version represented both with one constructor. We show that singleton symmetry then turns abstract membership into a false path equality and yields a closed stuck program.

Second, dependent-pair subtyping is conservative. The first-component type may be widened separately; the dependent member may change only while the first component is a singleton, with an explicit comparison after opening at that path. This rule supports the intended abstraction examples and supplies the evidence used by a direct semantic proof. It

κ	$::=$	$\star \mid \iota$
p, q	$::=$	$x \mid p.1 \mid p.a$
T, U	$::=$	$\top \mid \perp \mid (x : S) \rightarrow T \mid \langle x : S; a : \tau^\kappa \rangle$ $\mid \{p\} \mid p.A$
$\tau^\star$	$::=$	T
τ^ι	$::=$	$L..U$
$\delta^\star$	$::=$	z
δ^ι	$::=$	T
t, u	$::=$	$p \mid \lambda(x : S).t \mid \langle y; a = \delta^\kappa \rangle$ $\mid p.q \mid \text{let } x = t \text{ in } u \mid (t : T)$
v	$::=$	$\lambda(x : S).t \mid \langle y; a = \delta^\kappa \rangle$
Γ	$::=$	$\emptyset \mid \Gamma, x : T$

Figure 1. Source syntax. Uppercase labels conventionally denote type members.

is not known whether the unrestricted rule is unsound after singletons and selections are separated.

Contributions. The paper gives a closed counterexample showing that abstract membership cannot justify singleton symmetry; a calculus that separates those notions and retains dependent abstract members with explicit bound checks; and a direct finite-run safety proof based on exact stores, co-resolution of paths, and proof-relevant semantic coercions. The complete development is formalized in Lean. The language deliberately omits recursive objects, mutation, intersections, and unions; declarative transitivity remains primitive.

2 The calculus

2.1 Syntax

Figure 1 gives the syntax. Kinds distinguish term members ($\star$) from type members (ι). Accordingly, a generalized type is either a proper type or an interval, and a definition stores either a variable or a proper type.

The binder x in a function scopes over its codomain. In $\langle x : S; a : \tau \rangle$, it scopes over the member signature τ . We write $\tau[p/x]$ for capture-avoiding opening. Terms are in monadic normal form: pair components are variables, and both sides of an application are paths.

The four source judgments are

$$\Gamma \vdash p :: \tau^\kappa \quad \Gamma \vdash \tau_1^\kappa <: \tau_2^\kappa \quad \Gamma \vdash \tau^\kappa \text{ wf} \quad \Gamma \vdash t : T.$$

The double colon denotes *precise path typing*: unlike term typing, it does not contain subsumption.

2.2 Static semantics

The upper half of figure 2 defines precise path typing. Selection opens a dependent member with the receiver's first projection. If the outer pair carries a different label, P-Miss

continues at $p.1.a$; this is the static counterpart of missed-label lookup in figure 5.

Figure 3 gives the complete declarative subtyping relation. Besides the standard rules, three points matter below. S-ALIAS accepts only a term singleton; the selection rules retain an explicit nonempty-bounds premise; and dependent-pair subtyping is split between changing the first component and changing the member.

The well-formedness rules enforce the same distinctions. A singleton must contain a term-denoting path, whereas a selection must name an interval member. Forming or widening an interval requires a derivation $L <: U$; the lower bound varies contravariantly and the upper bound covariantly. These guards rule out locally inconsistent bounds, although the counterexample in section 3 will use only the interval $\top.. \top$.

In S-PAIR-SINGLE, the last premise compares the two members after opening them at p ; it is not another formation check. Declarative transitivity composes this rule with S-PAIR-FST. The lower half of figure 2 gives the complete well-formedness relation.

2.3 Term typing

Figure 4 gives term typing. Paths first receive singleton types, and dependent application substitutes the argument path into the result. Pair introduction is exact: the first component has type $\{y\}$, while a term member receives $\{z\}$ and a type definition receives $T..T$. The result of a let expression may not mention its bound variable; values that escape a let do so through stable store locations rather than through the source binder.

2.4 Dynamics

A store σ is an immutable sequence of values, and a stored value may refer only to earlier cells. We write $\sigma(x) = v$ for lookup and $\sigma, x \mapsto v$ for extension at a fresh location.

Source path resolution, written $\sigma \vdash p \Downarrow x$, is big-step. Projection returns the stored first component, a matching term member returns its stored location, and a missed label continues from the first component. Resolution is deterministic but partial on arbitrary stores; totality for well-typed paths is part of the soundness argument.

Machine states are triples $\langle \sigma, K, t \rangle$, with continuations made only of let frames. The rules at the bottom of figure 5 resolve applications and paths, evaluate let-bound computations, and allocate syntactic values. Store extension weakens the old continuation, written $K^\dagger$, into the extended scope. Application first resolves both operand paths and then requires an abstraction at the operator location; this is the case for which canonical forms are essential.

A state is final when its continuation is empty and its term is either a valid store location or a syntactic value. The initial state of t is $\text{initial}(t) = \langle \emptyset, [], t \rangle$.

3 Two design decisions

3.1 Singletons and abstract selections

The term singleton $\{p\}$ asserts value identity. In particular, S-ALIAS is justified when precise typing says that one path has another path's singleton type. An abstract selection $p.A$, by contrast, denotes an unknown type between the bounds stored at A . Membership in $p.A$ says nothing by itself about path identity.

In a precursor to λ_p , both forms used one type constructor, so singleton symmetry accepted either role. Let

$$\begin{aligned} f &= \lambda(x : \top).x, \\ q &= \langle f; A = \top \rangle, \\ h &= \lambda(z : q.A).q. \end{aligned}$$

The historical rules type the body of h by the chain

$$\{q\} <: \top <: q.A <: \{z\}.$$

The last edge applies singleton symmetry to the context binding $z : q.A$, treating abstract membership as equality. The same rules accept f as an argument of type $q.A$. Thus hf is assigned $\{f\}$, although it returns q .

Consequently, the closed program

```
let f = λ(x : ⤴).x in
let q = ⟨f; A = ⤴⟩ in
let h = λ(z : q.A).q in
let r = hf in rf
```

reaches an application whose operator location contains the pair q . It is neither final nor reducible. Every interval in the derivation is $\top.. \top$, so this is not a bad-bounds example. The failure also does not hinge on primitive transitivity: an admissible transitivity lemma would compose the same chain.

In the present syntax, a binding $z : q.A$ cannot be a premise for S-ALIAS; the historical derivation and execution are machine checked separately.

The counterexample shows that singleton symmetry must distinguish term identity from abstract membership. Separate constructors make this distinction explicit, although a suitably guarded symmetry rule would be another possible repair.

3.2 The pair rule

The unrestricted rule one might first write is

$$\frac{\Gamma \vdash S <: S' \quad \Gamma, x : S \vdash \tau <: \tau'}{\Gamma \vdash \langle x : S; a : \tau \rangle <: \langle x : S'; a : \tau' \rangle} \quad (\text{Unrestricted Pair})$$

Its semantic interpretation must work for the actual run-time first component y . In particular, a coercion between the pair types must later turn a realization of $\tau[y/x]$ into one of $\tau'[y/x]$. The induction hypothesis can construct such a coercion once y and its realization are known, but a finite coercion for the pair rule must be built before that run-time

$$\begin{array}{c}
\frac{\Gamma(x) = T}{\Gamma \vdash x :: T} \text{P-VAR} \quad \frac{\Gamma \vdash p :: \langle x:S; a:\tau \rangle}{\Gamma \vdash p.1 :: S} \text{P-FST} \quad \frac{\Gamma \vdash p :: \langle x:S; a:\tau \rangle}{\Gamma \vdash p.a :: \tau[p.1/x]} \text{P-SEL} \quad \frac{\Gamma \vdash p :: \langle x:S; b:\tau' \rangle \quad \Gamma \vdash p.1.a :: \tau \quad a \neq b}{\Gamma \vdash p.a :: \tau} \text{P-MISS} \\
\hline
\frac{}{\Gamma \vdash \perp \text{ wf}} \text{W-BOT} \quad \frac{}{\Gamma \vdash \top \text{ wf}} \text{W-TOP} \quad \frac{\Gamma \vdash p :: T}{\Gamma \vdash \{p\} \text{ wf}} \text{W-SINGLE} \quad \frac{\Gamma \vdash p :: \langle x:S; A:L..U \rangle}{\Gamma \vdash p.A \text{ wf}} \text{W-SELECT} \\
\frac{\Gamma \vdash S \text{ wf} \quad \Gamma, x:S \vdash T \text{ wf}}{\Gamma \vdash (x:S) \rightarrow T \text{ wf}} \text{W-FUN} \quad \frac{\Gamma \vdash S \text{ wf} \quad \Gamma, x:S \vdash \tau \text{ wf}}{\Gamma \vdash \langle x:S; a:\tau \rangle \text{ wf}} \text{W-PAIR} \quad \frac{\Gamma \vdash L \text{ wf} \quad \Gamma \vdash U \text{ wf} \quad \Gamma \vdash L <: U}{\Gamma \vdash L..U \text{ wf}} \text{W-BOUNDS}
\end{array}$$

Figure 2. Precise path typing (above) and well-formedness (below).

$$\begin{array}{c}
\frac{}{\Gamma \vdash \tau <: \tau} \text{S-REFL} \quad \frac{\Gamma \vdash \tau_1 <: \tau_2 \quad \Gamma \vdash \tau_2 <: \tau_3}{\Gamma \vdash \tau_1 <: \tau_3} \text{S-TRANS} \quad \frac{}{\Gamma \vdash \perp <: T} \text{S-BOT} \quad \frac{}{\Gamma \vdash T <: \top} \text{S-TOP} \quad \frac{\Gamma \vdash p :: T}{\Gamma \vdash \{p\} <: T} \text{S-WIDEN} \\
\frac{\Gamma \vdash p :: \{q\}}{\Gamma \vdash \{q\} <: \{p\}} \text{S-ALIAS} \quad \frac{\Gamma \vdash p.A :: L..U \quad \Gamma \vdash L <: U}{\Gamma \vdash L <: p.A} \text{S-SEL-LO} \quad \frac{\Gamma \vdash p.A :: L..U \quad \Gamma \vdash L <: U}{\Gamma \vdash p.A <: U} \text{S-SEL-HI} \\
\frac{\Gamma \vdash S' <: S \quad \Gamma, x:S' \vdash T <: T'}{\Gamma \vdash (x:S) \rightarrow T <: (x:S') \rightarrow T'} \text{S-FUN} \quad \frac{\Gamma \vdash S <: S'}{\Gamma \vdash \langle x:S; a:\tau \rangle <: \langle x:S'; a:\tau \rangle} \text{S-PAIR-FST} \\
\frac{\Gamma \vdash p :: P \quad \Gamma, x:\{p\} \vdash \tau <: \tau' \quad \Gamma \vdash \tau[p/x] <: \tau'[p/x]}{\Gamma \vdash \langle x:\{p\}; a:\tau \rangle <: \langle x:\{p\}; a:\tau' \rangle} \text{S-PAIR-SINGLE} \quad \frac{\Gamma \vdash L' <: L \quad \Gamma \vdash U <: U' \quad \Gamma \vdash L <: U}{\Gamma \vdash L..U <: L'..U'} \text{S-BOUNDS}
\end{array}$$

Figure 3. Generalized subtyping. Premises and conclusions have the same kind.

$$\begin{array}{c}
\frac{\Gamma \vdash p :: T}{\Gamma \vdash p : \{p\}} \text{T-PATH} \quad \frac{\Gamma, x:S \vdash t:T \quad \Gamma \vdash S \text{ wf}}{\Gamma \vdash \lambda(x:S).t : (x:S) \rightarrow T} \text{T-ABS} \quad \frac{\Gamma \vdash p : (x:S) \rightarrow T \quad \Gamma \vdash q : S}{\Gamma \vdash p q : T[q/x]} \text{T-APP} \\
\frac{\Gamma(y) = S \quad \Gamma(z) = T}{\Gamma \vdash \langle y; a=z \rangle : \langle x:\{y\}; a:\{z\} \rangle} \text{T-PAIR} \quad \frac{\Gamma(y) = S \quad \Gamma \vdash T \text{ wf}}{\Gamma \vdash \langle y; A=T \rangle : \langle x:\{y\}; A:T..T \rangle} \text{T-TYPE-PAIR} \\
\frac{\Gamma \vdash s:S \quad \Gamma \vdash T \text{ wf} \quad \Gamma, x:S \vdash t:T \quad x \notin \text{fv}(T)}{\Gamma \vdash \text{let } x = s \text{ in } t : T} \text{T-LET} \quad \frac{\Gamma \vdash t:T \quad \Gamma \vdash T \text{ wf}}{\Gamma \vdash (t:T) : T} \text{T-ASCRIIBE} \\
\frac{\Gamma \vdash t:S \quad \Gamma \vdash S <: T \quad \Gamma \vdash T \text{ wf}}{\Gamma \vdash t:T} \text{T-SUB}
\end{array}$$

Figure 4. Term typing. Names bound by pair types are in scope in their member signature.

$$\begin{array}{c}
\frac{}{\sigma \vdash x \Downarrow x} \text{R-VAR} \quad \frac{\sigma \vdash p \Downarrow x \quad \sigma(x) = \langle y; a=\delta \rangle}{\sigma \vdash p.1 \Downarrow y} \text{R-FST} \quad \frac{\sigma \vdash p \Downarrow x \quad \sigma(x) = \langle y; a=z \rangle}{\sigma \vdash p.a \Downarrow z} \text{R-HIT} \\
\frac{\sigma \vdash p \Downarrow x \quad \sigma(x) = \langle y; b=\delta \rangle \quad a \neq b \quad \sigma \vdash y.a \Downarrow z}{\sigma \vdash p.a \Downarrow z} \text{R-MISS} \\
\hline
\frac{\sigma \vdash p \Downarrow f \quad \sigma \vdash q \Downarrow y \quad \sigma(f) = \lambda(x:S).t}{\langle \sigma, K, p q \rangle \longrightarrow \langle \sigma, K, t[y/x] \rangle} \text{E-APP} \quad \frac{\sigma \vdash p \Downarrow x \quad p \text{ not a variable}}{\langle \sigma, K, p \rangle \longrightarrow \langle \sigma, K, x \rangle} \text{E-PATH} \quad \frac{F = \text{let } x = [] \text{ in } t}{\langle \sigma, K, \text{let } x = s \text{ in } t \rangle \longrightarrow \langle \sigma, F :: K, s \rangle} \text{E-LET} \\
\frac{F = \text{let } x = [] \text{ in } t}{\langle \sigma, F :: K, y \rangle \longrightarrow \langle \sigma, K, t[y/x] \rangle} \text{E-RETURN} \quad \frac{F = \text{let } x = [] \text{ in } t \quad v \text{ value} \quad \ell \text{ fresh}}{\langle \sigma, F :: K, v \rangle \longrightarrow \langle \sigma[\ell \mapsto v], K^\uparrow, t[\ell/x] \rangle} \text{E-ALLOC} \quad \frac{}{\langle \sigma, K, (t:T) \rangle \longrightarrow \langle \sigma, K, t \rangle} \text{E-ASCRIIBE}
\end{array}$$

Figure 5. Big-step path resolution (above) and CK-machine transitions (below).

component is available. Storing an inhabitant-indexed function in the coercion would make the realization relation occur negatively in its own mutual definition.

S-PAIR-SINGLE fixes a path p and supplies that specialized comparison explicitly. If the stored first component y has type $\{p\}$, then y and p co-resolve. Run-time conversion transports the opened comparison at p to the member opened at y .

The two pair rules still support the intended abstraction pattern. If $\Gamma \vdash p :: P$, then

$$\begin{aligned} &\langle x:\{p\}; A:\{p\}..\{p\} \rangle \\ &<: \langle x:\{p\}; A:\perp..\{x\} \rangle \\ &<: \langle x:P; A:\perp..\{x\} \rangle. \end{aligned}$$

The first step hides an exact definition behind dependent bounds while the first component is a singleton; the second widens that component.

This is a proof-motivated conservative restriction, not a demonstrated boundary of soundness. We have no counterexample to the unrestricted rule after singleton types and abstract selections have been separated.

4 Soundness

Write $\text{initial}(t) \longrightarrow^* s$ when the initial machine state of t reaches s in finitely many steps. The source-facing result is the usual non-stuckness theorem.

Theorem 4.1 (Finite-run type safety). *If $\emptyset \vdash t : T$ and $\text{initial}(t) \longrightarrow^* s$, then s is final or there exists s' such that $s \longrightarrow s'$.*

The theorem ranges over finite executions and does not assert termination. Preservation is proved for a stronger run-time invariant rather than by extending the source term judgment to machine states.

Write $\Delta \Vdash_{\text{exact}} s : U$ when the store of s is recorded at exact introduction types and its current term and continuation satisfy the structural run-time judgments. Write $(\Gamma, T) \preceq (\Delta, U)$ when Δ extends Γ by allocations and U is the corresponding weakening of T .

Theorem 4.2 (Finite-run preservation). *If $\emptyset \vdash t : T$ and $\text{initial}(t) \longrightarrow^* s$, then for some Δ, U ,*

$$(\emptyset, T) \preceq (\Delta, U) \quad \text{and} \quad \Delta \Vdash_{\text{exact}} s : U.$$

The changing context is bookkeeping forced by allocation: a fresh location extends the store context, and the result type is weakened along that extension. The exact structural judgment is the mathematical preservation invariant; [theorem 4.1](#) is its source-facing corollary.

4.1 Path resolution and exact stores

The first obstacle is path lookup. The tempting statement that resolution preserves the precise synthesized type is false. Suppose a stored pair has first component y . The path $p.1$ may synthesize $\{y\}$, whereas the endpoint y synthesizes

whatever type the store context assigns to that location. These types are related, but not literally equal.

The proof instead separates totality from preservation. For the term fragment of precise path typing, an exact well-typed store gives

$$\frac{\Gamma \vdash p :: T \quad \text{ExactStore}(\Gamma, \sigma)}{\exists x. \sigma \vdash p \Downarrow x.}$$

If p has ordinary term type T and resolves to x , then x has ordinary type T . The induction follows the same hit and miss structure as P-SEL and P-MISS; ordinary subsumption absorbs the change from the path's precise type to the context type of its endpoint.

Exactness is essential for the next step. An ordinary store typing may record a location at any supertype of its stored value, hiding the information needed by both lookup and canonical forms. The lookup lemmas above are therefore stated directly for exact stores. From a location viewed as a function one must recover an abstraction and its original signature. An exact store records the syntax-directed type assigned by T-ABS, T-PAIR, or T-TYPE-PAIR. Allocation extends the store with precisely this type; value inversion and narrowing recheck the suspended let body under the more informative binding.

4.2 Co-resolution and structural typing

Reduction also changes the spelling of paths. Let $p \equiv_{\sigma} q$ be the least congruence containing paths that resolve to the same location in σ . Types admit conversion along this equivalence. For example, a selected member opened at $p.1$ must be related to the same member opened at the location returned by the projection. Co-resolution supplies exactly that equality.

The source judgments embed into proof-only structural judgments for paths, subtyping, terms, continuations, and states. They expose the source constructors and add conversion by $\equiv_{\sigma}$. This layer is not an extension of the source calculus: its purpose is to state the run-time invariant and to make the conversion steps used by preservation explicit. A representative rule is replacement of co-resolving paths inside a type:

$$\frac{p \equiv_{\sigma} q}{\Gamma \Vdash \tau[p/x] <: \tau[q/x]}. \quad (\text{Conversion})$$

Because $\equiv_{\sigma}$ is a congruence, the same principle applies beneath projections, selections, functions, and pair signatures.

4.3 Realization and semantic coercions

Source evaluation returns locations, but the proof must also account for type members. It therefore uses generalized endpoints

$$e ::= \text{val}(x) \mid \text{type}(W).$$

On term paths, generalized resolution agrees with the source relation. A value endpoint realizes a proper type; a type

endpoint realizes an interval. The defining clauses needed below are as follows. A location realizes a function or pair type only if the store contains the corresponding value form. It realizes $\{p\}$ when it co-resolves with p . A stored definition W realizes $L..U$ together with evidence taking realizers of L to realizers of W , and realizers of W to realizers of U .

That evidence is represented by a finite, proof-relevant semantic coercion $\tau_1 \Rightarrow_s \tau_2$. A coercion acts on endpoints:

$$\frac{\tau_1 \Rightarrow_s \tau_2 \quad e \in \llbracket \tau_1 \rrbracket_{\Gamma, \sigma}}{e \in \llbracket \tau_2 \rrbracket_{\Gamma, \sigma}}. \quad (\text{Coercion action})$$

Identity and composition interpret reflexivity and transitivity. Selection uses the two coercions stored with an interval realization. Function coercions retain the residual domain and codomain comparisons later needed by beta reduction. Crucially, coercions are finite syntax rather than semantic functions, keeping the mutually defined realization relation strictly positive. The explicit opened premise of S-PAIR-SINGLE supplies the finite member coercion that the unrestricted pair rule lacks.

4.4 Fundamental lemma and canonical forms

The central induction is a simultaneous substitution argument. A semantic path substitution ρ from Γ to Δ must satisfy three conditions: each binding is mapped to a structurally checked path at its substituted type; ρ maps the source structural path relation into $\equiv_\sigma$; and each mapped variable resolves to an endpoint realizing its substituted context type. Under such a substitution, path checking and subtyping have the following common interpretation.

Lemma 4.3 (Fundamental lemma). *Suppose ρ realizes Γ in an exact store (Δ, σ) .*

1. *If $\Gamma \Vdash p :: \tau$, then $\rho(p)$ resolves to an endpoint realizing $\rho(\tau)$.*
2. *If $\Gamma \Vdash \tau_1 <: \tau_2$, then $\rho(\tau_1) \Rightarrow_s \rho(\tau_2)$.*

The cases for selection use interval realization; transitivity composes coercions. In the singleton-pair case, the first component co-resolves with the fixed path p , so run-time conversion transports the explicit opened coercion to the actual component. Instantiating lemma 4.3 with the identity substitution gives generalized path resolution and a semantic coercion for every structural subtyping derivation in an exact store.

Applying such a coercion to the realization of a location singleton yields the canonical forms used by the machine.

Lemma 4.4 (Canonical forms). *Let σ be exact for Γ .*

1. *If $\Gamma \Vdash \{x\} <: (z : S) \rightarrow T$, then $\sigma(x)$ is an abstraction.*
2. *If $\Gamma \Vdash \{x\} <: \langle z : S; a : \tau \rangle$, then $\sigma(x)$ is a pair.*

The function case also recovers the exact closure signature and the residual subtypings relating it to $(z : S) \rightarrow T$. Those residuals are what make dependent beta reduction type preserving.

4.5 Progress and preservation

Progress is by induction on structural typing of the current term and continuation. In an application, path resolution supplies locations for the operator and operand, and lemma 4.4 exposes an abstraction at the operator. Projection and selection use the pair part of canonical forms and the hit/miss correspondence between static and dynamic lookup.

Preservation is by cases on the machine step. If p normalizes to x , co-resolution permits structural conversion from $\{p\}$ to $\{x\}$ inside the run-time invariant; the ordinary endpoint theorem above explains why literal preservation of precise typing was the wrong statement. Beta reduction uses the closure signature recovered by the function coercion. When a let-bound value is allocated, exact value inversion determines the new store binding, narrowing checks the suspended body at that exact type, and the old continuation is weakened into the extended scope. The remaining administrative steps preserve the invariant directly. Iterating preservation and applying progress at the endpoint proves theorem 4.1.

The calculus, counterexample, and safety proof are mechanized in Lean 4 [2].

5 Related work

DOT combines abstract type members with recursive objects, refinements, intersections, and a rich subtyping lattice [1, 5]. Its soundness proofs must control which abstract bounds can be exposed in arbitrary contexts. The tight-typing account of Rapoport et al. [3], for example, factors reasoning through inert contexts in which run-time values admit strong inversion principles.

λ_p has no recursive self types or intersections and does not use an inert/tight-typing split. The exact store invariant provides the narrower inversion fact needed here: a location retains the introduction type of its stored function or pair even when declarative subtyping presents it at another type. Semantic coercions interpret the remaining subtyping derivation, including primitive transitivity and abstract selection, rather than trying to normalize it to a syntax-directed derivation.

pDOT adds arbitrary-length paths and singleton types to DOT and gives a Coq-mechanized safety proof [4]. It is the closest point of comparison for path equality and abstract selection. In λ_p , record extension is instead represented by dependent pair chains, and both static selection and dynamic lookup have an explicit missed-label case. The present calculus asks which obligations remain when this path mechanism is separated from recursive objects, intersections, and unions.

6 Conclusion

λ_p isolates paths, dependent pairs, and abstract type members. Its counterexample shows that abstract membership

cannot justify singleton symmetry. Separating the two notions and conservatively restricting pair subtyping yields a direct finite-run safety proof. Whether unrestricted pair subtyping is sound for the syntax-separated calculus remains open.

References

- [1] Nada Amin, Tiark Rompf, and Martin Odersky. 2014. Foundations of Path-Dependent Types. In *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages and Applications*. Association for Computing Machinery, New York, NY, USA, 233–249. doi:10.1145/2660193.2660216
- [2] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction – CADE 28 (Lecture Notes in Computer Science, Vol. 12699)*, André Platzer and Geoff Sutcliffe (Eds.). Springer International Publishing, Cham, 625–635. doi:10.1007/978-3-030-79876-5_37
- [3] Marianna Rapoport, Ifaz Kabir, Paul He, and Ondřej Lhoták. 2017. A Simple Soundness Proof for Dependent Object Types. *Proceedings of the ACM on Programming Languages* 1, OOPSLA, Article 46 (2017), 27 pages. doi:10.1145/3133870
- [4] Marianna Rapoport and Ondřej Lhoták. 2019. A Path to DOT: Formalizing Fully Path-Dependent Types. *Proceedings of the ACM on Programming Languages* 3, OOPSLA, Article 145 (2019), 29 pages. doi:10.1145/3360571
- [5] Tiark Rompf and Nada Amin. 2016. Type Soundness for Dependent Object Types (DOT). In *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications*. Association for Computing Machinery, New York, NY, USA, 624–641. doi:10.1145/2983990.2984008